

Deep Networks

From Shallow to Feedforward, and Universal Approximation

Prof. Nirav Bhatt

5 August

Outline

Composing Networks

Where the shallow network left us

- A shallow network is a linear read-out over **learned features**
 $h_d = a[\theta_{d0} + \theta_{d1}x]$ (Classes 5–6)
- With enough hidden units it is a **universal approximator** — but the number of units may be enormous
- Question for today: what if we feed the output of one network into the **input of another**?

Stacking shallow networks gives a **deep network**. The same building block, composed — and far more efficient.

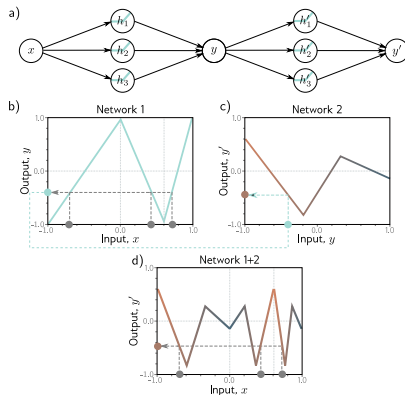
An intuition: solve a big problem in stages

- A shallow network must map raw inputs to the answer in **one leap** — every hidden unit works directly on x
- A deep network solves the problem in **stages**: layer 1 extracts simple features, layer 2 combines them into richer ones, and so on
- Reading is the analogy: **strokes** $\rightarrow$ **letters** $\rightarrow$ **words** $\rightarrow$ **meaning**. Nobody jumps from ink to meaning in a single step
- Each stage is *easy* because it builds on the stage before — the difficulty is **spread across depth**

SHALLOW one leap: raw input $\rightarrow$ answer

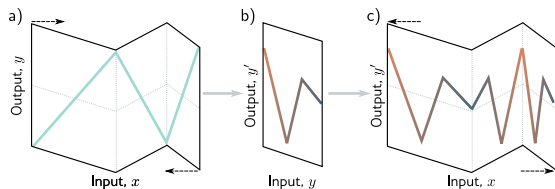
DEEP staged: input $\rightarrow$ features $\rightarrow$ features of features $\rightarrow$ answer

Composing two networks



Prince (2023), Fig. 4.1: network 1 (left) feeds network 2 (right). Each is piecewise linear; their composition is piecewise linear with *more* joints.

The folding analogy



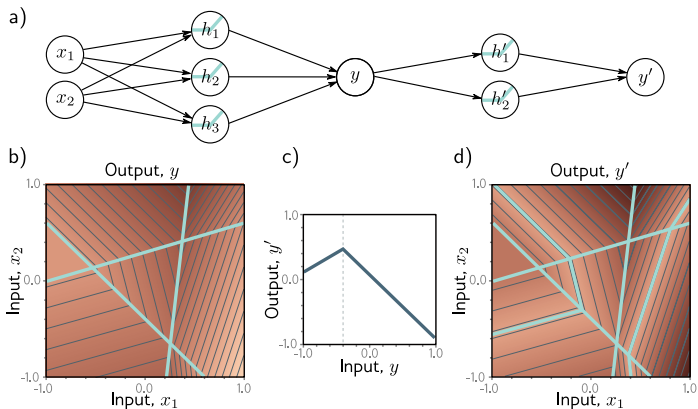
Prince (2023), Fig. 4.3.

- The first network **folds** input space back onto itself
- The second acts on the folded space — one of its regions maps to **several** input regions
- Folding is why depth multiplies regions instead of merely adding them

SHALLOW joints add: D units $\rightarrow D+1$ regions

DEEP folding *multiplies* regions across layers

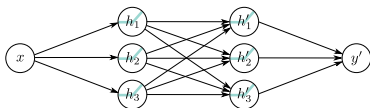
Composing in two dimensions



Prince (2023), Fig. 4.2: composing networks with a 2D input; the second network re-uses the first network's convex regions to build a more intricate partition.

The Deen Network

Two hidden layers, one equation



Prince (2023), Fig. 4.4.

Layer 1, layer 2, then output:

$$h_1 = a[\theta_0 + \theta_1 x]$$

$$h_2 = a[\psi_0 + \psi_1 h_1]$$

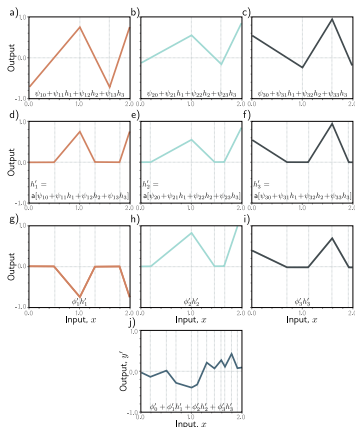
$$y = \phi_0 + \phi_1 h_2$$

Each layer: **linear map** then **activation**.

SHALLOW one hidden layer

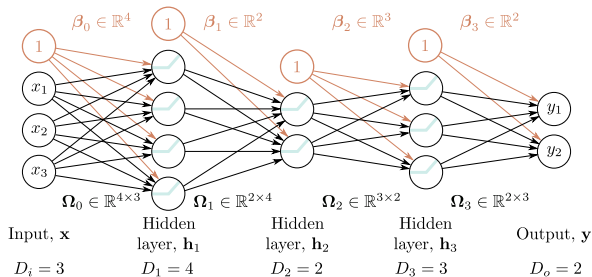
DEEP hidden layers *stacked*: same block, composed

Computed step by step



Prince (2023), Fig. 4.5: the two-layer network's output, built up layer by layer.

The general deep network



Prince (2023), Fig. 4.6.

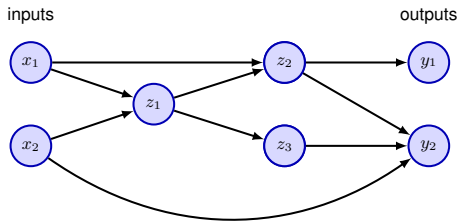
K hidden layers, matrix notation:

$$\mathbf{h}_k = \mathbf{a}[\beta_{k-1} + \Omega_{k-1} \mathbf{h}_{k-1}]$$

- Weights Ω_k , biases β_k per layer
- **Depth** = number of hidden layers; **width** = units per layer
- Both are **hyperparameters** — chosen, not learned

Prince (2023), Eqs. 4.15–4.16.

The same object in Bishop's notation



Bishop & Bishop (2024), Fig. 6.15.

- A **feedforward network**: information flows input $\rightarrow$ output, no cycles
- Each layer applies $\mathbf{z}^{(l)} = h(\mathbf{W}^{(l)}\mathbf{z}^{(l-1)})$
- “deep” simply means **more than one** hidden layer Bishop & Bishop, §6.1

SHALLOW single-layer NN / shallow net

DEEP multi-layer / **feedforward** network (FNN)

Building depth recursively

The two-layer construction **extends recursively** (Bishop & Bishop, 2024, §6.2):

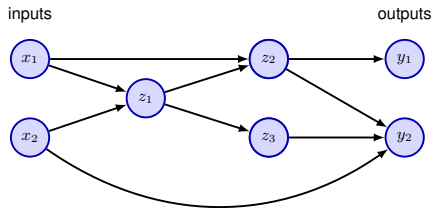
$$\mathbf{z}^{(1)} = h(\mathbf{W}^{(1)}\mathbf{x}), \quad \mathbf{z}^{(2)} = h(\mathbf{W}^{(2)}\mathbf{z}^{(1)}), \quad \dots, \quad \mathbf{y} = f(\mathbf{W}^{(L)}\mathbf{z}^{(L-1)})$$

- Each layer feeds the next — the **same operation** (linear map + nonlinearity) repeated
- The construction extends recursively into a **hierarchical model with many layers** — the basis for deep networks Bishop & Bishop, §6.2
- Nothing new is needed — depth is **repetition of a single idea**

SHALLOW $\mathbf{y} = \phi_0 + \phi h(\mathbf{W}\mathbf{x})$

DEEP $\mathbf{y} = f(\mathbf{W}^{(L)}h(\dots h(\mathbf{W}^{(1)}\mathbf{x})))$

A note on counting layers



Bishop & Bishop (2024), Fig. 6.15.

The same network is described inconsistently in the literature:

- “three-layer” — counting *all* layers of units, inputs included
- “single-hidden-layer” — counting only *hidden* layers
- Bishop recommends counting **layers of weights** (adaptive parameters)

Bishop & Bishop, §6.3

We will say a network is **deep** when it has **more than one** hidden layer.

What Deep Networks

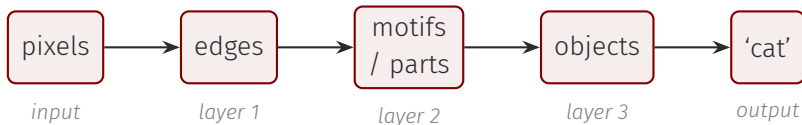
Depth as an inductive bias

- Universality tells us *what* a network *can* represent; it says nothing about what it *prefers* to learn
- A deep architecture encodes an **inductive bias**: that the output relates to the input through a **hierarchy** of intermediate concepts Bishop & Bishop, §6.3.1
- For images, the map from pixels to a concept like ‘cat’ is **highly nonlinear** — hopeless for one layer, natural for many
- The right bias means the network needs **less data** and generalises better

SHALLOW bias: output is one nonlinear map of the input

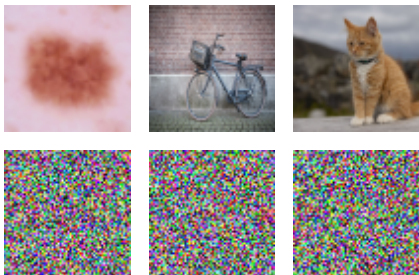
DEEP bias: output is a **hierarchy** of simpler maps

Hierarchical representations



- Early layers detect **low-level** features (edges, corners); later layers assemble them into **high-level** concepts
- Each layer reuses the layer below — features are **shared** across everything built on top of them
- This compositional reuse is what one-layer networks cannot exploit Bishop & Bishop, §6.3.1

Why hierarchy is the right bias for real data



Bishop & Bishop (2024), Fig. 6.8: natural images
(top) vs. random pixels (bottom).

- Almost every random pixel pattern is **noise** — real images are vanishingly rare among all possible images
- Natural data lives on a **low-dimensional manifold** with rich structure
- A hierarchy of features is exactly how to capture that structure efficiently — **compositional data**, **compositional model**

Distributed representations



fur



whiskers



round eyes



pointy ears

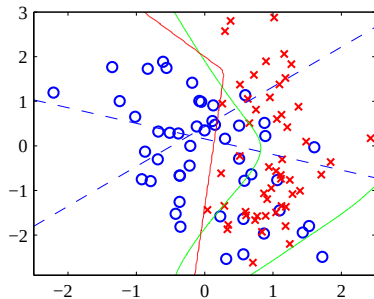
a hidden layer

- Each unit signals whether a **feature** is present (high) or absent (low)
 - M units name M features — but their **combinations** can encode up to 2^M distinct patterns
 - Concepts are represented by **patterns of activity**, not single units — a **distributed code**
- Bishop & Bishop, §6.3.2

SHALLOW features fixed and hand-chosen

DEEP features learned, and **shared combinatorially**

Representation learning



Bishop & Bishop (2024), Fig. 6.11.

- Each layer **transforms** the data to make the task easier
 - By the final hidden layer, the classes are arranged so a **simple linear classifier** separates them
 - That layer's output is the **learned representation** (embedding); the last layer just draws a line
- Bishop & Bishop, §6.3.3

SHALLOW one transform, then a linear read-out

DEEP many learned transforms, then a linear read-out

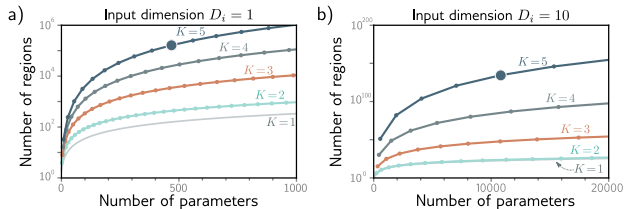
Why Denth?

Universal approximation, revisited

- A shallow network is **already** a universal approximator (Class 5) — so why go deep?
- Because universality says a solution **exists**, not that it is **small**: matching a complex function may need exponentially many hidden units in one layer
- Deep networks can represent the same functions with **far fewer parameters**

The question is not *whether* but *how efficiently* a network represents a function.

Linear regions per parameter



Prince (2023), Fig. 4.7.

- Max linear regions vs. number of parameters
- For the same parameter budget, **deeper** networks create *far* more regions
- Depth grows expressivity **multiplicatively**; width, only additively

SHALLOW regions grow *polynomially* with width

DEEP regions grow *exponentially* with depth

Shallow vs. deep: the trade-off

	Shallow (1 layer)	Deep (many layers)
Expressivity	universal, but needs many units	universal with far fewer units
Regions	polynomial in width	exponential in depth
Features	one level of learned features	hierarchical features, layer on layer
Training	convex read-out only	harder landscape; needs backprop (Class 11+)

Depth trades a harder optimisation problem for a dramatic gain in representational efficiency.

Why depth tends to win in practice

- Real data is **compositional**: edges $\rightarrow$ shapes $\rightarrow$ objects; phonemes $\rightarrow$ words $\rightarrow$ meaning
- Deep networks mirror this — each layer builds **features of the previous layer's features**
- Fewer parameters for the same function means **better generalisation** and less data to fit
- Caveat: depth alone is not enough — training deep networks needs the tools of the coming classes (initialisation, normalisation, residual connections)

SHALLOW flat: features of the raw input

DEEP hierarchical: features of features of features

Takeaways

1. A **deep network** is shallow networks **composed**: each layer is a linear map followed by an activation
2. Composition **folds** input space, so regions **multiply** across layers rather than add
3. Both shallow and deep nets are universal — but deep nets reach the same functions with **exponentially fewer parameters**
4. Depth suits **compositional** data by building hierarchical features — at the cost of a harder optimisation problem

Next: depth and width efficiency theorems, then how to actually **train** these networks (backpropagation).

References

- S. J. D. Prince. *Understanding Deep Learning*. MIT Press, 2023. Chapter 4. Figures 4.1–4.7 and teaching sequence from the author’s repository (<https://udlbook.github.io/udlbook/>), CC BY-NC-ND, with attribution.
- C. M. Bishop & H. Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024. Chapter 6. Figure 6.15 from the authors’ published figure set, with attribution.
- G. Cybenko (1989); K. Hornik (1991); Montufar et al. (2014) on regions and depth.

Questions?